Zusatztrommeln des C 8205Z – Zeitmodell und Modulspezifikation

Version: 0.01-009

Eigenes Projekt C 8205Z, nicht Teil des C-8205-Simulators (Interpreter, Assembler). Rekonstruktion; was festgelegt und was Annahme ist, steht in Abschnitt 8. Rechnung: `zt/zt_zeitmodell.py`.

1. Eckdaten

Größe	Haupttrommel (HT)	Zusatztrommel (ZT), 4 Stück
Worte	4096 zu 33 Bit (128 Spuren zu 32 Sektoren)	je 4096, gleich aufgebaut
Umlauf	3,328 ms (104 µs je Position)	20 ms (625 µs je Sektor, 3000 U/min)
Faktor	1	6,01
Gruppenbefehl (G, GX, W, WX)	alle 32 Positionen in einem Umlauf	–

Position = physischer Platz auf der Spur (0..31, zeitlich aufeinanderfolgend). Logischer Sektor = Sektornummer in der Adresse. Zuordnung (oktal):

``` HT Position 0..31: 00 15 32 07 24 01 16 33 10 25 02 17 34 11 26 03 20 35 12 27 04 21 36 13 30 05 22 37 14 31 06 23 ZT Position 0..31: 00 10 20 30 15 25 35 05 32 02 12 22 07 17 27 37 24 34 04 14 01 11 21 31 16 26 36 06 33 03 13 23 ```

Beide Folgen sind Permutationen von 00..37. Die ZT-Folge ist die spaltenweise Lesung der HT-Folge (4 Zeilen zu 8).

2. Blöcke

- Ein Block = 8 Worte. Die Spur hat 4 Blöcke, Block k = HT-Positionen 8k..8k+7 = ZT-Positionen p ≡ k (mod 4). Der Zusammenhang ist `ZT[4i+k] = HT[8k+i]`: ZT-Position p = 4i + k trägt das Wort i des Blocks k (geprüft in `pruefe_folgen`).
- Daher haben HT und ZT bei jedem Sektor die gleiche logische Nummer: Sektor n der HT-Spur ↔ Sektor n der ZT-Spur. Die Nummerierung der ZT sorgt dafür, daß alle 32 Zellen einer Spur auf den GLEICHEN Sektornummern liegen.
- HT-Zeit je Block: Gruppenbefehl; Warten auf die Position 8k (0..3,33 ms) plus 8 · 0,104 = 0,83 ms, also höchstens 4,16 ms.
- ZT-Zeit je Block: 8 Worte im Abstand von 4 Sektoren (2,5 ms); der Block braucht 7 · 2,5 + 0,625 = 18,125 ms, also höchstens einen Umlauf. Der Block darf an jeder seiner 8 Positionen beginnen (der Puffer nimmt die Worte in beliebiger Reihenfolge); die Anlaufwartezeit beträgt deshalb höchstens 2,5 ms.

3. Ablauf eines E/A-Befehls (Eingabe 661: ZT → HT; Ausgabe 601: HT → ZT)

Ein Befehl überträgt IMMER eine ganze Spur (32 Worte, 4 Blöcke). Der Rechner wartet bis zum Ende des Befehls; es läuft kein zweiter Befehl auf der Trommel.

Im Befehl arbeiten zwei Seiten gegeneinander, mit zwei alternierenden Puffern zu je 8 Worten (Puffer 0 und 1) und je einem Zugriffssemaphor. Der Semaphor wirkt nur innerhalb des Befehls: Der Initiator (die Seite, die den Puffer füllt) setzt ihn, der Empfänger (die Seite, die den Puffer leert) gibt ihn frei.

Eingabe 661: Initiator = ZT, Empfänger = HT. Ausgabe 601: Initiator = HT, Empfänger = ZT.

Block n (n = 0..3 in der Reihenfolge des Befehls) benutzt Puffer n mod 2:

1. Der Initiator wartet, bis der Semaphor des Puffers frei ist, und setzt ihn.
2. Der Initiator füllt den Puffer (Eingabe: ZT-Block lesen; Ausgabe: HT-Block mit Gruppenbefehl lesen).
3. Er meldet den Puffer voll.
4. Der Empfänger schreibt den Puffer (Eingabe: HT-Block mit Gruppenbefehl; Ausgabe: ZT-Block) und gibt den Semaphor frei.
5. Der Initiator beginnt mit Block n+1 im anderen Puffer, sobald dessen Semaphor frei ist, noch während der Empfänger Block n schreibt.

Der Befehl endet, wenn der Empfänger den letzten Block geschrieben hat (alle Semaphore frei).

Blockreihenfolge: Sie legen die Lage der HT- und der ZT-Adresse fest. Modell: Es beginnt der Block k, dessen Position p ≡ k (mod 4) als nächste an der ZT-Kopf kommt; dann folgen k+1, k+2, k+3 (mod 4). Zwischen
```

Blockende und Beginn des nächsten Blocks liegt eine Umschaltzeit u (Vorgabe 0,3 ms), danach beginnt der nächste Block an der ersten passenden Position.

4. Zeiten (aus dem Modell, 64 × 64 Phasen)

Größe	Rechnung	Ergebnis
ZT-Block	7 · 2,5 + 0,625	18,125 ms
Spur Eingabe 661, Doppelpuffer	u = 0,3 ms	80,9 / 82,8 / 84,7 ms (min / Mittel / max)
Spur Ausgabe 601, Doppelpuffer	u = 0,3 ms	82,5 / 83,8 / 85,6 ms
Spur ohne Doppelpuffer (Vergleich)	Eingabe / Ausgabe	81,4 / 84,3 / 87,5 und 82,5 / 85,1 / 88,1 ms
Angabe	4,2 + 4 · 20	85 ms (Obergrenze)
Einzeladresse (Lesen, Ändern, Schreiben)	20 + 3,33 + 20	43,3 ms (Angabe 44)
Einzeladresse, beide Trommeln synchron aktuell	3,33 + 20	23,3 ms (Angabe 24)

Abhängigkeit von der Umschaltzeit u (Eingabe, Maximum): 0 ms → 77,1 ms; 0,3 bis 2,5 ms → 84,6 ms (gleich, weil der nächste Block immer an der nächsten Position p ≡ k beginnt); ab etwa 3 ms → 92,0 ms.

Vergleichsmodell „fester Start“ (der Block beginnt erst an Position k, bis 20 ms Anlauf): Spur 81,0 / 92,5 / 103,9 ms (`START = 'fest'`, `ORDNUNG = 'fest'`). Es übersteigt die Angabe 85 ms und gilt daher nicht.

5. Befehle des C 8205Z

Befehl	Wirkung
`5k00 030` (k = 1..4)	Anwahl der Zusatztrommel k (Festlegung des Inhabers, korrigiert 30.10.2026: 5100 = ZT1 (Trommel 0) … 5400 = ZT4 (Trommel 3)); gilt bis zur nächsten Anwahl
`zzz0 bbbb 601`	Ausgabe: HT-Spur bbbb → gewählte ZT, Spur zzz0 (Befehlswort 11 Oktalziffern; zzz0 = obere vier Ziffern, bisher ohne Bedeutung)
`zzz0 bbbb 661`	Eingabe: gewählte ZT, Spur zzz0 → HT-Spur bbbb

Der Adreßtransport `aaaa 032` entfällt; die ZT-Adresse steht im selben Befehlswort. Adreßteil zzz0: wie die Ausgabe-Adresse des R-300-Kodes (vier Oktalstellen, die letzte „x“ ist beliebig und wird ignoriert). Wirksam ist nur die Spur: `spur = (adresse >> 5) & 0x7F`. Der Sektorteil ist ohne Bedeutung, es gibt keinen Anfangsblock.

Fehlerfälle: Beide Trommeln haben 128 Spuren und es wird immer eine ganze Spur übertragen; Adreß- und Kapazitätsfehler gibt es nicht. Der E/A-Befehl kommt immer NACH der Anwahl (wie die Ausgabe auf SW). Fehlt die Anwahl, verhält er sich wie ein Befehl ohne angewähltes Gerät: Halt mit Flag A (Ausgabe-Halt bzw. Eingabe-Halt).

6. Modulspezifikation (C-Modul als Hintergrundprozess)

Ziel: Ein eigenständiges Modul `zt_modul` (C, wie `c-module/trommel.c`), das die vier Zusatztrommeln samt den zwei Puffern als Hintergrundprozess simuliert und mit wenigen Aufrufen an eine Haupttrommel angekoppelt wird.

6.1 Konstanten und Tabellen

```c
#define ZT_ANZAHL 4
#define ZT_SPUREN 128
#define ZT_SEKTOREN 32
#define ZT_SEKTORZEIT_NS 625000 /* eine ZT-Position */
#define ZT_BLOCK 8 /* Worte je Block */
#define ZT_PUFFER 2
#define ZT_UMSCHALT_NS 300000 /* u, Vorgabe 0,3 ms, einstellbar */
static const unsigned char HT_PERM[32] = {
000,015,032,007,024,001,016,033, 010,025,002,017,034,011,026,003, 020,035,012,027,004,021,036,013,
030,005,022,037,014,031,006,023 };
static const unsigned char ZT_PERM[32] = {
000,010,020,030,015,025,035,005, 032,002,012,022,007,017,027,037, 024,034,004,014,001,011,021,031,
016,026,036,006,033,003,013,023 };
```

6.2 Zustand

- Je ZT: Speicher 4096 Worte zu 33 Bit (Datei im Format der HT: `aaaa: ttttttttttt`, `aaaa` = logische Adresse Spur · 32 + Sektor), Position `zt_pos` (0..31), die sich alle 625 µs weiterschaltet (aus der Monotonic-Uhr abgeleitet, wie bei der HT, Startlage zufällig oder fest einstellbar).
- Gemeinsam: gewählte Trommel (0 = keine, 1..4), `puffer[2][8]` (uint64), `sem[2]` (FREI/BESETZT) und `voll[2]`.
- Nur eine ZT wird gleichzeitig benutzt; die Position der anderen ist nicht beobachtbar.

6.3 Schnittstelle

```c int zt_einschalten(const char *pfad[ZT_ANZAHL], int start_index); / start_index < 0: zufällig / void zt_ausschalten(void); int zt_anwahl(int k); / 5k00 030, k = 1..4 */ int zt_eingabe(unsigned int zzz0, unsigned int bbbb, zt_stat_t *s); / zzz0 bbbb 661: ZT-Spur = (zzz0 >> 5) & 0x7F; blockiert bis zum Ende der Spur */ int zt_ausgabe(unsigned int zzz0, unsigned int bbbb, zt_stat_t *s); / zzz0 bbbb 601 */ void zt_ht_ankoppeln(const zt_ht_t *ht); / HT-Anbindung, Vorgabe: trommel.h / void zt_abbrechen(void); / weckt den wartenden Aufrufer */

typedef struct { uint64_t (*lesen)(unsigned adr); / Vorgabe trommel_lesen */ int (*schreiben)(unsigned adr, uint64_t w); / Vorgabe trommel_schreiben */ unsigned (*aktueller_sektor)(void); } zt_ht_t; / Vorgabe trommel_aktueller_sektor */ typedef struct { unsigned long long dauer_ns; unsigned long long ticks_ht; unsigned wartezeit_block[4]; } zt_stat_t; ```

Rückgaben: `ZT_OK` 0; `ZT_KEINE_ANWAHL` 1 (der Aufrufer löst den Halt mit Flag A aus); `ZT_ABGEBROCHEN` 2; `ZT_FEHLER` 3 (Datei, ZT nicht eingeschaltet).

Kopplung an die HT: In der echten Maschine ist der Puffer Elektronik (8 × 33 Bit Flip-Flops); die Übertragung Puffer ↔ HT läuft im Takt der HT-Elektronik (104 µs je Wort, wie im Gruppenbefehl) und ist zeitlich festgelegt, ein Zeitproblem gibt es dort nicht. Im Simulator darf der Block daher als EINE Einheit behandelt werden: Das Modul liest und schreibt die 8 Worte eines HT-Blocks über die Direktzugriffe `trommel_lesen` / `trommel_schreiben` (ohne Koinzidenz, logische Adresse `(spur << 5) | HT_PERM[8k + i]`) und rechnet die Zeit selbst: Wartezeit bis zur HT-Position 8k (aus der aktuellen Sektornummer `trommel_aktueller_sektor()`) plus 8 · 104 µs. Einzelne `..._warten`-Aufrufe je Wort sind nicht nötig und würden bei Verzögerung des Aufrufers einen Umlauf (3,33 ms) kosten; eine Änderung an `trommel.c` ist nicht erforderlich. Da der Rechner im E/A-Befehl wartet, gibt es keinen Zugriffskonflikt um die HT. Die Struktur `zt_ht_t` enthält deshalb `lesen(adr)` und `schreiben(adr, wert)` (Vorgabe: `trommel_lesen`, `trommel_schreiben`) sowie `aktueller_sektor()`.

### 6.4 Ablauf (beide Richtungen)

`` zt_eingabe/zt_ausgabe(b): wenn gewählt == 0: return ZT_KEINE_ANWAHL t0 = jetzt; folge[0..3] = blockfolge(t0) /* Abschnitt 3 */ Initiator-Thread und Empfänger-Thread starten (oder ein Thread mit zwei Zuständen) Initiator, für n = 0..3: Empfänger, für n = 0..3: warte bis sem[n%2] == FREI warte bis voll[n%2] sem[n%2] = BESETZT schreibe puffer[n%2] (Block folge[n]) fülle puffer[n%2] (Block folge[n]) voll[n%2] = 0 voll[n%2] = 1 sem[n%2] = FREI warte bis Empfänger fertig; return ZT_OK ``

Block k füllen/schreiben: Seite ZT: warte bis `zt_pos mod 4 == k` (und seit dem letzten Blockende mindestens `u` vergangen); danach 8 Zugriffe, jeder alle 4 Positionen (2,5 ms); Wort i im Puffer ↔ ZT-Position `4i + k` ↔ logischer Sektor `ZT_PERM[4i + k]`. Der Block endet 18,125 ms nach seinem Beginn. Seite HT: Block als Einheit auf den Positionen 8k..8k+7 (Zeit: Warten bis Position 8k, dann 0,83 ms), Wort i ↔ HT-Position `8k + i` ↔ logischer Sektor `HT_PERM[8k + i]` (gleich `ZT_PERM[4i + k]`).

### 6.5 Zeitmodell im Modul

Das Modul soll auch rein rechnerisch ohne Threads arbeiten können (virtuelle Zeit): `zt_modell_dauer(phase_zt, phase_ht, umschalt, richtung)` ist die C-Fassung der Funktionen `spur_ein`/`spur_aus` aus `zt_zeitmodell.py`. Damit sind Tests unabhängig von der Rechnerlast möglich.

### 6.6 Umsetzung (C-Modul `zt_modul` 0.01-003, Ordner `C8205Z/zt_modul/`)

Dateien: `zt_modul.h`, `zt_modul.c`, `testframe_zt.c`, `Makefile` (`make test`, Linux; Windows-Zweig über `plattform.h` nicht übersetzt). Der Simulator (`c-module/`) bleibt unverändert; seine Quellen (`trommel.c`, `plattform.h`) werden von dort benutzt (`SIM=../../c-module`).

Wie in 6.3 bis 6.5 festgelegt: `zt_einschalten`, `zt_ausschalten`, `zt_anwahl`, `zt_eingabe(zzz0, bbbb, &s)`, `zt_ausgabe(zzz0, bbbb, &s)`, `zt_ht_ankoppeln`, `zt_abbrechen`, `zt_modell_dauer`. Der Aufrufer ist der Initiator, ein Hintergrundthread der Empfänger; zwei Puffer zu 8 Worten, je ein Semaphor, Block n im Puffer n mod 2. Der Plan aller Zeitpunkte steht vor dem Beginn des Befehls fest (`zt_modell_dauer` mit Plan).

Ergänzungen gegenüber 6.3:

| Funktion | Zweck |
|---|---|
| `zt_zeitmodus(ZT_ZEIT_VIRTUELL / ZT_ZEIT_ECHT)` | VIRTUELL (Vorgabe): nicht schlafen, `dauer_ns` liefert die Modelldauer; ECHT: beide Seiten warten bis zu den Zeitpunkten des Modells (Aufruf dauert 81 bis 86 ms) |
| `zt_umschaltzeit(ns)` | u, Vorgabe 0,3 ms |
| `zt_phasen_fest(an, phase_zt_ns, phase_ht_ns)` | feste Phasen für reproduzierbare Tests; `an = 0`: Phasen aus Uhr und `aktueller_sektor()` |
```

Funktion	Zweck
zt_lesen, zt_schreiben, zt_leeren, zt_gewaehlt, zt_semaphore_frei, zt_letzte_adresse (0.01-003)	Direktzugriffe ohne Zeit, Leeren (hebt die Anwahl auf), Beobachtung

Festlegungen der Umsetzung (Annahmen, bitte prüfen):

- Zeiten in Ganzzahl-Nanosekunden; Modellzeit: HT-Position 0 beginnt bei 0 mod 3,328 ms, `phase_zt` = Beginn der ZT-Position 0 in Modellzeit.
- Gemessene Phasen: HT-Phase aus `aktueller_sektor()` mit der Genauigkeit einer Position (104 µs; Befehlsbeginn = Beginn dieser Position); ZT-Phase aus der Monotonic-Uhr ab Einschalten (`start_index` = ZT-Position beim Einschalten, < 0 zufällig).
- `zt_stat_t.wartezeit_block[n]` in µs: Anlaufwartezeit der Initiatorseite (Eingabe: ZT, Ausgabe: HT) für den n-ten Block des Befehls. `dauer_ns` ist die Gesamtzeit einschließlich Hintergrund (Ende des letzten Blocks).
- Die Daten werden beim Füllen (Initiator) bzw. am Blockende (Empfänger) bewegt, die Zeit nur im Modus ECHT abgewartet. ZT-Dateien (`zt1..zt4`, Format der HT) werden beim Einschalten geladen (fehlende angelegt), nach jeder Ausgabe und beim Ausschalten gesichert.
- `zt_abbrechen` wirkt nur auf einen laufenden Befehl (Ergebnis `ZT_ABGEBROCHEN`, Semaphore danach frei, teilweise übertragene Blöcke bleiben). Ein zweiter gleichzeitiger Aufrufer erhält `ZT_FEHLER`.

Prüfung (`make test`, 153 Prüfungen): Permutationen (`ZT[4i+k] = HT[8k+i]`); die fünf Referenzzeilen aus Abschnitt 7 (auf 0,5 µs); Statistik über 64 × 64 Phasen (Eingabe 80,912 / 82,784 / 84,708 ms, Ausgabe 82,500 / 83,767 / 85,578 ms); Abhängigkeit von u (77,1 / 84,6 / 92,0 ms); Datenübertragung mit Ersatz-HT in beide Richtungen, ZT getrennt, andere Spuren unverändert, Semaphore frei, acht Phasenlagen; Persistenz; Echtzeit (rund 84 ms gemessen) und Abbruch; Kopplung an die echte HT (`trommel.c`).

6.7 Anbindung an GRAFIK (Erweiterungsmodul `c8205_ztmodul`, `grafik_zt.py`)

Der Python-Interpreter (TEST) rechnet die Zusatztrommeln mit Zeit 0 (`c8205_zt.py`, wie die Haupttrommel dort); das C-Modul mit Zeitmodell gehört an den GRAFIK-Interpreter. Aufruf: `c8205_grafik.py --C8205Z[=VERZ]`; die Dateien `zt1.txt` bis `zt4.txt` (Format der HT) liegen in `VERZ` (Vorgabe `Trommel/ZT/`) und sind mit denen des Python-Moduls austauschbar.

- `C8205Z/zt_modul/python/c8205_ztmodul.c` bindet `zt_modul.c` mit `-DZT_OHNE_TROMMEL` ein (kein zweites `trommel.c`, es gibt nur die eine Haupttrommel in `c8205_trommel`). Die HT wird über drei Rückrufe angekoppelt: `c8205_trommel.lesen`, `.schreiben` (Direktzugriffe ohne Koinzidenz, wie in 6.3 vorgesehen) und `.aktueller_sektor`. Die Rückrufe laufen im Aufrufer- und im Empfängerthread und nehmen die GIL; blockierende Aufrufe geben sie frei. Bauen: `cd C8205Z/zt_modul/python && python3 setup.py build_ext --inplace` (liefert `c8205_ztmodul.abi3.so`; `SIM` = Ordner `c-module`, Vorgabe `../../../c-module`).
- Zeit je nach `--trommelzeit` der Haupttrommel: `alt` (Wanduhr): Modus ECHT, Phasen aus Uhr und `aktueller_sektor()`; der Befehl dauert in Echtzeit 81 bis 86 ms. `echt` und `schnell` (virtuelle Zeit `vt`, Wortzeit 104 µs): Modus VIRTUELL mit festen Phasen aus `vt` (HT-Phase = `(vt mod 32) · 104 µs`, ZT-Phase aus der Startlage `idx0` und `vt`); die Dauer wird als Ticks auf `vt` aufgeschlagen, bei `echt` schläft `_pace()` sie ab, bei `schnell` nicht.
- Ohne Anwahl: Halt mit Flag A (601) bzw. Flag E (661), wie in TEST. Abbruch (X im Fenster, Netz-Aus) beendet einen laufenden Befehl mit `GrafikAbbruch`.
- Anzeige im HW-Check (`--HWCHECK --C8205Z`, `grafik_hwcheck.py` HW-005, `GrafikZT.ansicht()` G5-002): Je Zusatztrommel eine Zeile mit der zuletzt bearbeiteten Adresse (Spur Sektor; `zt_letzte_adresse(k)`, laeuft waehrend eines Befehls mit; `--- --` = noch keine), hinter der angewaehlten ZT ein kleiner gruener Punkt, dazu ein einziges Inhaltsfeld <<ZTk>> fuer die angewaehlte ZT (es ist immer nur eine aktiv). Entscheidung des Inhabers 25.10.2026 (Variante: zuletzt bearbeitete Adresse). Pruefung: `C8205Z/Tests_Z/zt_hwcheck_test.py` (mit Tk-Attrappe).
- Prüfung: `C8205Z/Tests_Z/zt_grafik_tests.sh` vergleicht TEST und GRAFIK (Haupttrommel und `zt1..zt4` Byte für Byte, Endregister, Halt ohne Anwahl) für `zt-001` bis `zt-003` in allen drei Trommelzeiten: alle gleich. Das Pult-Fenster ist nicht geprüft.

7. Referenzwerte für Tests (`zt_zeitmodell.py`, u = 0,3 ms, `START = 'fliegend'`, `ORDNUNG = 'zuerst'`)

Phase der ZT `pz` (ms, Verschiebung gegen die HT, Position 0 beginnt bei `pz`), Beginn des Befehls `t0` (ms ab HT-Umlaufbeginn); Dauer in ms (Gesamtdauer des Befehls):

pz	t0	Blockfolge	Eingabe 661	Ausgabe 601
0	0	0 1 2 3	83,200	82,500
0,3	0	0 1 2 3	83,200	82,800
5,0	1,0	2 3 0 1	83,864	82,750
12,34	2,5	1 2 3 0	81,532	82,965
19,9	3,3	2 3 0 1	81,564	82,850

Weitere Prüfungen: `pruefe_folgen()` (ZT[k::4] = HT[8k..8k+7]); über alle Phasen Eingabe 80,9 / 82,8 / 84,7 ms und Ausgabe 82,5 / 83,8 / 85,6 ms (Abschnitt 4); jede Eingabe gefolgt von der Ausgabe auf dieselbe Spur reproduziert die Daten; nach dem Befehl sind alle Semaphore frei.

8. Festlegungen, Annahmen, offen

Festgelegt: Ein E/A-Befehl überträgt immer eine ganze Spur (32 Worte); der Rechner wartet im Befehl, die Pufferarbeit läuft innerhalb des Befehls im Hintergrund; es läuft nie ein zweiter Befehl auf der Trommel; je Puffer ein Zugriffssemaphor (Initiator setzt, Empfänger gibt frei), nur innerhalb des Befehls wirksam; zwei Puffer zu je 8 Worten; Adresse wie die R-300-Ausgabeadresse; Blockreihenfolge ergibt sich aus der Lage der Adressen; es gibt keine Adreßfehler; Anwahl vor dem E/A-Befehl, sonst Halt wie bei einem Gerät ohne Anwahl; Zuordnung 5100 = ZT1 ... 5400 = ZT4; Anbindung an den Interpreter c8205_interp.py (Simulator) versuchsweise mit dem Schalter `--C8205Z` (Python-Modul `c8205_zt.py`, Zeit 0). Befehlscodes 601/661 (Wiederholungsbefehl W) statt 401/461.

Festlegung: Mindestabstand der ZT-Zugriffe g = 4 Sektoren. Gründe: (1) nur g = 4 macht „jede 4. ZT-Position = eine HT-Achtergruppe“; (2) g ≥ 5 paßt nicht (7 · 5 + 1 = 36 > 32 Sektoren, mehr als ein Umlauf); (3) ein kleineres g bräuchte schnellere Elektronik, als bei gleicher Bauart und Einzeltransistoren zu erwarten ist.

Annahmen: ZT und HT sind unsynchron (zufällige Phase); die Umschaltzeit u = 0,3 ms liegt im Bereich 0 < u ≤ 2,5 ms, in dem das Ergebnis kaum von u abhängt; der Block darf an jeder seiner 8 ZT-Positionen beginnen; die HT-Blöcke werden wie mit einem Gruppenbefehl in einem Durchlauf übertragen; der Simulator behandelt sie als Einheit mit berechneter Zeit.

Offen: Pult-Matrix links am Z-Pult (Fotos aus der Technischen Sammlung Dresden).